

Astar 2025 Training: Resource Management & Optimization Homework

Introduction

This homework contains 3 main problems.

For each problem, you will first be asked to implement some function based on given framework and then ask some extra questions to guide you to think deeper about the problems.

There will be several unit test (implemented with Google Test) for each problem. After you finish the implementation, you should run the unit tests to check if your implementation is correct.

Note that passing all unittest might not mean your code is 100% correct (e.g. there are some concurrency problems that are difficult to trigger). If you are not confident with your code, you can search solutions online since the problems are very common.

Setup

The following steps are only for **Linux**, if you don't have access to any linux machine or you don't want to waste time on finding a linux environment, you can figure out how to use CMake to build the project on your own machine by searching google or asking GPT. The code used are all in C++ standard, so it should be easy to build on any platform. But be aware that a high version of C++ (maybe above C++ 20) is required for building the project.

To start with, make sure you are at the root directory of this project (`HomeworkHandout/`). Then follow the steps below:

1. `mkdir build`
2. `cmake ..`
3. `make`

4. Run any test by `./src/<name>/<name>` , where you should replace `<name>` with the test you want to run, including `custom_string` , `custom_barrier` and `safe_hashmap` .

Hint

- These problems are designed to be closely related to the topics we have covered in the class.
- If not specified, you can modify any code (including headers) except the unittest code (`unittest.cpp` in each subfolder) to solve the problems.
- Although the problems may seem complex, the solution is usually simple and elegant. To your reference, my implementations for each functions in Q1 and Q2 are no more than 10 lines of code. For Q3 the length is no more than 20 lines.
- There might be some modern C++ features that you are not familiar with. Search for and study them because most of them are very easy but useful.

Submission Requirement

Finish the code and make sure you passed all the unittests.

Answer the extra questions in a text file (in any format, txt, pdf, etc.). Short explanation is enough, but make sure you have thought about the questions.

Push the completed code and a text file to the branch you have used in RMTraining github repo.

Problems start from here.

Q1: Custom String

This problem is to test your OOP skills as well as your knowledge about memory management and move semantics in C++.

Implementation Requirement

The given code framework is under `src/custom_string` folder.

Note: You should see that some code is disabled in `custom_string.h` and `custom_string.cpp` by a `CopyAndSwap` macro. See the detail of this in "Extra Requirement" section.

In this problem, you are required to implement a custom string class `CustomString` which is a simple wrapper of `char *`. It should have the following features (for detailed definition, see in `custom_string.h`):

1. A default constructor that initializes the string to an empty string.
2. A constructor that initializes the string with a given `const char*`.
3. A destructor that will properly release the resources.
4. Copy constructor and copy assignment operator that will copy the string from another `CustomString` object.
5. Move constructor and move assignment operator that will move the string from another `CustomString` object.
6. A `swap(CustomString &str_)` **member function** that will swap the content of current object and `str_`.
7. Member functions `length()`, `to_string()`, and `c_str()`, which returns the length of the string, the string as a `std::string` object, and the raw `char *` of the string, respectively.
8. Overloaded operators `+` and `+=` that will concatenate two `CustomString` objects.
9. Overloaded operators `==` that will compare two `CustomString` objects.
10. Overloaded operators `[]` that can both access and modify the character of the string.

Note: in terms of memory management, you should keep the memory space of different

`CustomString` objects independent. That is, if you modify the string of one object, it should not affect any other object.

Extra Questions

The disabled code in `custom_string.h` and `custom_string.cpp` is a technique called Copy-and-Swap idiom. This idiom is invented to avoid tricky implementation of copy and move assignment operators.

Search for and study this idiom, and then set the `CopyAndSwap` macro to `1` in `custom_string.h` and implement the merged version of copy and move assignment operators in `custom_string.cpp`.

After you pass the test with Copy-and-Swap idiom, please consider the following question:

- In this problem, which implementation is more efficient, the one with Copy-and-Swap idiom or the one without it? Why? Hint: Consider the number of memory allocation, deallocation and copy.

Q2: Custom Barrier

This problem is to test your knowledge about multi-threading, concurrency issue and synchronization in C++.

Implementation Requirement

The given code framework is under `src/custom_barrier` folder.

In class, we have introduced 2 basic synchronization method: barrier and condition variable. We have discussed that condition variable is more basic and flexible than barrier. So in this problem, you are required to implement a custom barrier class `CustomBarrier` using condition variable.

The `CustomBarrier` class should have the following features (for detailed definition, see in `custom_barrier.h`):

1. A constructor that initializes the barrier with a given number of threads.
2. A `wait()` member function that will block the current thread until all threads reach the barrier.

Note: Think carefully about the concurrency issue and possible deadlock. There could be some hidden problems in your implementation that are not triggered by the unittest cases.

Extra Questions

This is a relatively open question. Situation could be different on different machines.

Look into the unittest code and find the last test case which is related to KMeans algorithm in `custom_barrier/unittest.cpp`.

Read and compare the 2 versions (serial and parallel) of implementation.

How many thread are used in the parallel version? Then how many times of the performance improvement do you expect to see?

Then run the test and you should see a performance report in the output. Does the reported result match your expectation? If not, what could be the reason?

Hint: Think about the relationship between threads and physical CPU cores.

Bonus: Based on your thinking, modify the number of threads in the parallel version and see if the performance is improved.

Q3: Fast Safe Hashmap

This is an open problem, you can implement it in any method you like as long as you keep the functions normal and improve the performance.

This problem is to encourage you to learn more advanced topics about locks (e.g. lock granularity) than we have covered in class and use them to optimize the performance of a concurrent data structure.

If you don't know what a hashmap is, basically you can think it as a dictionary in Python. The basic algorithm can be found [here](#).

Specifically, in this problem, we will use chaining to handle hash collision. That is, each bucket in the hashmap is a linked list of key-value pairs.

Implementation Requirement

The given code framework is under `src/safe_hashmap` folder. You will find an fully implemented version of a safe hashmap (`class SafeHashMap`).

`SafeHashMap` provides the basic functionalities of a hashmap, including `update` and `get` .

`update` member function will update the value of a key in the hashmap. If the key does not exist, a new key-value pair will be inserted.

`get` member function will return the value of a key in the hashmap wrapped in an `std::optional` . If the key does not exist, an empty `std::optional` will be returned.

Moreover `SafeHashMap` is thread-safe, which means it can be used in a multi-threading case for concurrent read/write. Its basic idea is to use a lock on the whole hashmap to avoid concurrency issue. This is a simple but not efficient way to handle concurrency.

Your task is to implement a more efficient version of `SafeHashMap` called `FastSafeHashMap` . The `FastSafeHashMap` should have the same feature as `SafeHashMap` (`update` , `get` , and thread-safe) but with better performance.

Note: For "thread-safety" details, you can look into the unittest code and see what kinds of concurrency issue are tested.

Hint: Think about the granularity of the lock. If you are still not satisfied, you can further consider concurrent read situation and think about how to optimize it.